

Decision Control Structures in C (if-else, switch, ternary)

📌 Definition to Remember

Decision control structures direct program execution along different execution paths based on the boolean evaluation of conditional expressions.

1. Concept Overview & Fundamentals 🗨️

Decision Control Structures in C (if-else, switch, ternary) forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Code is compiled directly into native machine instructions without virtual machine overhead.
- Memory is strictly partitioned into **Code/Text Segment**, **Data Segment (Initialized/BSS)**, **Stack**, and **Heap**.
- Direct pointer and register manipulation provides deterministic performance critical for operating systems, embedded hardware, and database kernels.

2. Technical Architecture & Memory Mechanics ⚙️

Memory Layout & Storage Organization

```
+-----+
|                STACK SEGMENT                |
| - Local variables, function call stack frames, parameters |
| - High memory growing downwards towards Heap |
+-----+
|
| v
| ^
|
+-----+
|                HEAP SEGMENT                |
| - Dynamic memory allocations via malloc(), calloc() |
| - Low memory growing upwards towards Stack |
+-----+
|                INITIALIZED DATA SEGMENT    |
| - Global and static variables with non-zero initial values |
+-----+
|                BSS SEGMENT (UNINITIALIZED)  |
| - Global and static variables initialized to zero |
+-----+
|                CODE / TEXT SEGMENT          |
| - Read-only executable binary instructions |
+-----+
```

Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers dynamic scalability	Stack: O(1) push/pop; Heap: O(1) alloc
Type Checking	Static Typing	Compiler verifies types during compilation	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memory address	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops, and call stacks	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Decision Control Structure (if-else, switch, goto, ternary)\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.
- **Embedded & IoT Devices:** Microcontrollers with constrained RAM (8KB-64KB) require C's predictable footprint.
- **Database Engine Kernels:** High-throughput storage engines (PostgreSQL, SQLite, MySQL InnoDB) rely on low-level buffer pool managers written in C.
- **Game Engine Foundations:** Graphics rasterization, physics engines, and GPU interfaces utilize C/C++ memory arrays for zero-overhead computation.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: AbsoluteTO

Problem Description & Objective:

Create a program to calculate the absolute value of a given integer using ternary operator.

C Code Implementation:

```
//Create a program to calculate the absolute value of a given integer using ternary operator.

#include <stdio.h>
int main() {
    /*
     * Logic:
     * - Goal: Create a program to calculate the absolute value of a given integer using ternary
operator.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num, absolute;

    printf("Enter number: ");
    scanf("%d", &num);

    absolute = (num < 0) ? -num : num;

    printf("Absolute value is %d\n", absolute);

    return 0;
}
```

Problem 2: CategorizeAgeGroup

Problem Description & Objective:

Implementation of CategorizeAgeGroup

C Code Implementation:

```

/*
Create a program that categorize a person into different age groups.

Child -> below 13
Teen -> below 20
Adult -> below 60
Senior -> above 60
*/
#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program that categorize a person into different age groups. Child -> below 13
    Teen -> below 20 Adult -> below 60 Senior -> above 60
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int age;

    printf("Enter age: ");
    scanf("%d", &age);

    if (age <= 13) {

        printf("Child\n");

    } else if (age <= 20) {

        printf("Teenager\n");

    } else if (age <= 60) {

        printf("Adult\n");

    } else {

        printf("Senior Citizen\n");
    }

    return 0;
}

```

Problem 3: GradeCalculator

Problem Description & Objective:

Implementation of GradeCalculator

C Code Implementation:

```

/*
Create a program that calculates grades based on marks.

A -> above 90%
B -> above 75%
C -> above 60%
D -> above 30%
F -> below 30%
*/

#include <stdio.h>
int main() {
    /*
    * Logic:
    * - Goal: Create a program that calculates grades based on marks. A -> above 90% B -> above 75% C ->
above 60% D -> above 30% F -> below 30%
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    float marks;

    printf("Enter your marks: ");
    scanf("%f", &marks);

    if (marks >= 90) {

        printf("Grade A\n");

    } else if (marks >= 75) {

        printf("Grade B\n");

    } else if (marks >= 60) {

        printf("Grade C\n");

    } else if (marks >= 30) {

        printf("Grade D\n");

    } else {

        printf("You got FAIL!!\n");

    }

    return 0;
}

```

Problem 4: GreatestNumber

Problem Description & Objective:

Create a program that determines the greatest of the three numbers.

C Code Implementation:

```
//Create a program that determines the greatest of the three numbers.

#include<stdio.h>
int main(){
    /*
     * Logic:
     * - Goal: Create a program that determines the greatest of the three numbers.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num1,num2,num3;

    printf("Enter 1st number: ");
    scanf("%d",&num1);
    printf("Enter 2nd number: ");
    scanf("%d",&num2);
    printf("Enter 3rd number: ");
    scanf("%d",&num3);

    if(num1 >= num2 && num1 >= num3){
        printf("1st number is greatest!!!");
    }else if(num2 >= num1 && num2 >= num3){
        printf("2nd number is greatest!!!");
    }else{
        printf("3rd number is greatest!!!");
    }

    return 0;
}
```

Problem 5: LeapYear

Problem Description & Objective:

Create a program that determines if a given year is a leap year (considering conditions like divisible by 4 but not 100, unless also divisible by 400).

C Code Implementation:

```
//Create a program that determines if a given year is a leap year (considering conditions like divisible
by 4 but not 100, unless also divisible by 400).

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Create a program that determines if a given year is a leap year (considering conditions
like divisible by 4 but not 100, unless also divisible by 400).
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int year;

    printf("Enter year: ");
    scanf("%d", &year);

    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {

        printf("%d is a Leap Year\n", year);

    } else {

        printf("%d is NOT a Leap Year\n", year);

    }

    return 0;
}
```

Problem 6: MinimumNumberTO

Problem Description & Objective:

Create a program to find the minimum of two numbers using ternary operator.

C Code Implementation:

```
//Create a program to find the minimum of two numbers using ternary operator.

#include<stdio.h>
int main(){
    /*
     * Logic:
     * - Goal: Create a program to find the minimum of two numbers using ternary operator.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num1 , num2;

    printf("Enter 1st number: ");
    scanf("%d",&num1);
    printf("Enter 2nd number: ");
    scanf("%d",&num2);

    (num1 < num2)?printf("1st is Minimum"):printf("2nd is Minimum");

    return 0;
}
```

Problem 7: NumberChecker

Problem Description & Objective:

Create a program that determines if a number is positive, negative, or zero.

C Code Implementation:

```
//Create a program that determines if a number is positive, negative, or zero.

#include<stdio.h>
int main(){
    /*
     * Logic:
     * - Goal: Create a program that determines if a number is positive, negative, or zero.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num;

    printf("\nEnter Number: ");
    scanf("%d",&num);

    if(num == 0){
        printf("\nNumber entered by you is Zero");
    }else if(num < 0){
        printf("\nNumber entered by you is Negative");
    }else{
        printf("\nNumber entered by you is Positive");
    }

    return 0;
}
```

Problem 8: OddEven

Problem Description & Objective:

Create a program that determines if a number is odd or even.

C Code Implementation:

```
//Create a program that determines if a number is odd or even.

#include<stdio.h>
int main(){
    /*
     * Logic:
     * - Goal: Create a program that determines if a number is odd or even.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num;
    printf("Enter number: ");
    scanf("%d",&num);

    if(num == 0){
        printf("\nNumber entered by you is ZERO!!!");
    }else if(num % 2 == 0){
        printf("\nNumber entered by you is EVEN!!!");
    }else{
        printf("\nNumber entered by you is ODD!!!");
    }

    return 0;
}
```

Problem 9: OddEvenTO

Problem Description & Objective:

Create a program to find if the given number is even or odd using ternary operator.

C Code Implementation:

```
//Create a program to find if the given number is even or odd using ternary operator.

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Create a program to find if the given number is even or odd using ternary operator.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int num;

    printf("Enter number: ");
    scanf("%d", &num);

    (num % 2 == 0) ? printf("Even\n") : printf("Odd\n");

    return 0;
}
```

Problem 10: PrintMonthSwitch

Problem Description & Objective:

Create a program to print the month of the year based on a number (1-12) input by the user.

C Code Implementation:

```
//Create a program to print the month of the year based on a number (1-12) input by the user.

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Create a program to print the month of the year based on a number (1-12) input by the
user.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int month;

    printf("Enter month number (1-12): ");
    scanf("%d", &month);

    switch(month) {

        case 1:
            printf("January\n");
            break;

        case 2:
            printf("February\n");
            break;

        case 3:
            printf("March\n");
            break;

        case 4:
            printf("April\n");
            break;

        case 5:
            printf("May\n");
            break;

        case 6:
            printf("June\n");
            break;

        case 7:
            printf("July\n");
            break;

        case 8:
            printf("August\n");
            break;

        case 9:
            printf("September\n");
            break;
```

```
    case 10:
        printf("October\n");
        break;

    case 11:
        printf("November\n");
        break;

    case 12:
        printf("December\n");
        break;

    default:
        printf("Invalid month number\n");
}

return 0;
}
```

Problem 11: ScoreCategoryTO

Problem Description & Objective:

Create a program to Based on a student's score, categorize as "High", "Moderate", or "Low" using the ternary operator (e.g., High for scores > 80, Moderate for 50-80, Low for < 50).

C Code Implementation:

```
//Create a program to Based on a student's score, categorize as "High", "Moderate", or "Low" using the
ternary operator (e.g., High for scores > 80, Moderate for 50-80, Low for < 50).

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Create a program to Based on a student's score, categorize as "High", "Moderate", or "Low"
using the ternary operator (e.g., High for scores > 80, Moderate for 50-80, Low for < 50).
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int score;

    printf("Enter student score: ");
    scanf("%d", &score);

    (score > 80)
        ? printf("High\n")
        : (score >= 50)
            ? printf("Moderate\n")
            : printf("Low\n");

    return 0;
}
```

Problem 12: SimpleCalculatorSwitch

Problem Description & Objective:

Create a program to create a simple calculator that uses a switch statement to perform basic arithmetic operations like addition, subtraction, multiplication, and division.

C Code Implementation:

//Create a program to create a simple calculator that uses a switch statement to perform basic arithmetic operations like addition, subtraction, multiplication, and division.

```
#include <stdio.h>
```

```
int main() {  
    /*  
    * Logic:  
    * - Goal: Create a program to create a simple calculator that uses a switch statement to perform  
    basic arithmetic operations like addition, subtraction, multiplication, and division.  
    * - Prompts the user for required inputs.  
    * - Executes standard control flow, conditions, or loops to compute the result.  
    * - Prints the formatted output to the console.  
    */
```

```
  
    int num1, num2;  
    char op;
```

```
  
    printf("Enter first number: ");  
    scanf("%d", &num1);
```

```
  
    printf("Enter operator (+, -, *, /): ");  
    scanf(" %c", &op);
```

```
  
    printf("Enter second number: ");  
    scanf("%d", &num2);
```

```
    switch(op) {  
  
        case '+':  
            printf("Result = %d\n", num1 + num2);  
            break;  
  
        case '-':  
            printf("Result = %d\n", num1 - num2);  
            break;  
  
        case '*':  
            printf("Result = %d\n", num1 * num2);  
            break;  
  
        case '/':  
            printf("Result = %.2f\n", (float)num1 / num2);  
            break;  
  
        default:  
            printf("Invalid Operator\n");  
    }  
  
    return 0;  
}
```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. if-else statements evaluate non-zero as true and zero as false in C.
2. switch statement evaluates integral/char expressions; each case requires break to prevent fall-through.
3. default case in switch handles unmatched values.
4. Ternary operator (?:) provides an inline 3-operand conditional expression.
5. Avoid goto statements to prevent spaghetti code and maintain structured control flow.

⚡ Quick Recall

Boolean Test -> if / else if / else -> switch(integral) + break -> Fallback default